

Robust Reinforcement Learning for Robot Control Across Different Observation Including Sensor Noise

Seung Hwan Jung, Chelroy Limas, Yuzhou Liu, Russell Roberts, Jonathan Cheng

Project Type: Research focused. We study whether RL policies trained with noise-aware methods (noise augmentation and drop) retain performance under unseen sensor corruption types in simulated robotic control tasks

What Are We Trying to Do?

RL policies for robotic control are typically trained in simulators with clean, noiseless observations, yet real robots must operate with imperfect sensor measurements. This mismatch causes policies to degrade when deployed on physical hardware and in real-world simulations.

We measure robustness as how well a policy maintains performance under corrupted observations, using average return and success rate. Corruption refers to noise added to state observations at varying types and intensities.

We train PPO or SAC policies in MuJoCo / DeepMind Control Suite (e.g., Reacher) under three conditions:

- **Clean:** no noise during training
- **Noise-augmented:** Gaussian noise or dropout during training
- **Roll-Drop-inspired:** dropout during rollouts, tuned by one parameter

Tools:

To study how observation noise affects RL policies, we need a reliable simulation environment and a flexible programming setup. Our tools are:

- **MuJoCo:** A general-purpose physics engine used for simulation, allowing us to model robotic control tasks and inject noise into observations.
- **Python:** Used to implement, train, and evaluate the model, with the MuJoCo library installed within Python.
- **NumPy:** Essential for injecting noise into observations. We'll use it for Gaussian noise, dropout masks, and other corruption types.

Problems We Identified

Current approaches to handling observation noise in RL for robotic control fall into three main categories:

- **Clean training:** Standard RL policies are trained using uncorrupted observations. While simple and commonly used in simulation, these policies may be sensitive to observation noise that appears during deployment.
- **Noise augmentation:** Noise is deliberately added to observations during training so that the policy learns to operate under imperfect measurements. However, this approach may depend on knowing the type of noise in advance, and robustness to one corruption may not transfer to another.
- **Dropout-based robustness:** Methods such as Roll-Drop introduce dropout during policy rollouts to improve robustness without explicitly modeling a specific sensor noise distribution. This may provide better generalization, but we want to investigate how it compares with direct noise augmentation across different observation corruptions.

The main problem we want to investigate is therefore whether robustness learned from a particular training method transfers to observation corruptions that differ from those encountered during training.

Feasibility / Scope

The project will be conducted entirely in simulation, so no physical robot, sensors, or additional laboratory equipment are required. MuJoCo / DeepMind Control Suite provides lightweight robotic control environments that can be run on personal computers, while established implementations of PPO or SAC

can reduce the amount of algorithm development required.

We will initially focus on one robotic control environment and a small number of observation corruption types to keep training and evaluation computationally manageable. If time and computing resources allow, we can extend the experiments to additional environments or corruption types. This allows us to obtain a complete baseline comparison first while keeping further experiments optional.

Research Questions

Our experiments will focus on the following research questions:

1. How does the performance of RL policies trained with clean observations change as the type and intensity of observation corruption increase?
2. Do noise-augmented and Roll-Drop-inspired training improve robustness compared with clean training?
3. Does robustness learned under one type of observation corruption generalize to unseen corruption types during evaluation?
4. Is there a trade-off between robustness under corrupted observations and performance under clean observations?

Evaluation

We will evaluate all trained policies under both clean and corrupted observations. Each corruption type will be tested at multiple intensity levels to measure how performance degrades as observation quality decreases. Possible test corruptions include Gaussian noise, observation dropout, and other simple perturbations to state observations.

For each training condition, we will measure:

- Average episodic return
- Task success rate
- Performance degradation as corruption intensity increases

We will compare performance on corruption types encountered during training with performance on unseen corruption types. Multiple evaluation episodes will be used for each condition to reduce the effect of randomness. The results will be summarized

using robustness curves showing policy performance as a function of corruption intensity.

If we are successful, what difference will it make?

If successful, this project will provide a systematic comparison of how different RL training strategies handle observation corruption in simulated robotic control. In particular, we hope to determine whether robustness-aware training methods can generalize beyond the specific type of corruption used during training.

The results could provide practical insight into whether simple approaches such as noise augmentation or dropout-based training are sufficient when the exact sensor corruption expected at deployments is unknown. This could help improve the reliability of policies trained in simulation before they are deployed in environments with imperfect sensor measurements.

Timeline (rough)

Phase 1 (1-2 weeks): Environment Setup - verify baseline clean training data, implement noise injection utilities. Create a working pipeline before training begins

Phase 2 (2-3 weeks): Implementation - Build three training conditions (clean, noise-augmented, Roll-Drop) and run small pilots to catch issues early

Phase 3 (3-5 weeks): Training - Parallel Training within 4 weeks (given hardware exists).

Phase 4 (1-2 weeks): Evaluation - Test policies on both seen and unseen data at different intensities. Compute robustness curves and metrics.

- If training is slow, reduce to 2 seeds or lower task complexity

Phase 5 (1-2 weeks): Writing - Draft, compile results, finalize by December 7, 2026